You said:

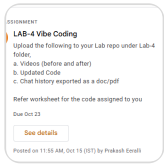


get me the url

ChatGPT said:

https://github.com/SETAPESU26/27_centipede ↗

You said:


ASSIGNMENT
LAB-4 Vibe Coding
Upload the following to your Lab repo under Lab-4 folder;
a. Videos (before and after)
b. Updated Code
c. Chat history exported as a doc/pdf
Refer worksheet for the code assigned to you
Due Oct 23
See details
Posted on 11:55 AM, Oct 15 (IST) by Prakash Gorati

Lab 4: VibeCoding
Objective
Use Vibe Coding tools to fix the broken code and add features to the same. Learn how
to prompt to get the job done in under three to four attempts. Clone a repo assigned
to you, make changes and push to your repo in your personal account.
THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!!
Duration
60 minutes

Software Requirements
● Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc)
Instructions:
1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one
by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo.

10.Have Separate Commits for each task
11.Do not raise PR to the main repo (SETAPESU26)

Deliverables:
Upload the following to your Lab repo under Lab-4 folder,

a. Videos (before and after)
b. Updated Code
c. Chat history exported as a doc/pdf

ChatGPT said:

Your Lab 4 assignment is about fixing a broken Python game/puzzle using a vibe-coding tool, adding the features listed in your assigned repository's README, and pushing the completed work to your personal GitHub repository.

You need to submit these three things under **Lab-4** in your lab repository:

1. **Videos:** 10 seconds before changes and 10 seconds after all changes.
2. **Updated code:** The completed Python project.
3. **Chat history:** Exported as a DOC or PDF, including your prompts and the AI's responses.

Important: make a **separate Git commit for each task**, and do not create a pull request to the original `SETAPESU26` repository.

Your assigned repository is:

https://github.com/SETAPESU26/27_centipede ↗

Let's do it step by step. First, open the repository and read its README. Send me the README content or a screenshot of it. I'll guide you through cloning, running the game, recording the original video, fixing the bugs, adding the required features, and pushing each task in a separate commit without skipping any deliverables.

You said:

ok

ChatGPT said:

Okay! 😊

Step 1: Open your assigned repository:

Open 27_centipede on GitHub ↗

1. Open the **README.md** file.
2. Read the instructions and feature requirements.
3. Send me a screenshot of the README or copy its contents here.

We'll proceed one step at a time. **Don't change any code yet**—we need to understand the requirements and record the original game before making changes.

You said:

Centipede Repair Lab

This project is a single-file Centipede-lite clone using **Pygame**. It introduces students to segment-chain movement, chain splitting, and projectile-vs-grid collision using a small, readable object-oriented codebase.

What's Provided

A working Centipede-lite game with:

- A player confined to the bottom band of the screen who fires straight up
- A centipede made of chained segments that winds back and forth across the screen, dropping a row whenever it hits a border or a mushroom
- A field of mushrooms that block the centipede's path and can be shot down
- Wave-based difficulty, lives, and scoring, including splitting the centipede into two independent chains when a body segment is hit

It has **one deliberate bug** and **three optional features** left as empty functions. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

pip install pygame

3. Run the game:

python game.py
Controls: Arrow keys to move, Space to fire, `R` to reset.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the mushroom durability bug

> A mushroom is supposed to survive four hits before it's cleared from the field. In the current build, mushrooms disappear after only three hits instead. Check the comparison in `hit_mushroom` against `MUSHROOM_HP` (which is defined as 4 near the top of the file).

Task 2: Implement `mushroom_color(hp)`

> Called once per mushroom per frame in `draw`, as `color = mushroom_color(hp) or (200 - (MUSHROOM_HP - hp) * 40, 80, 170)`. It receives the mushroom's remaining hit points (1 through `MUSHROOM_HP`) and should return an `(r, g, b)` color, or `None` to keep the default (which fades as the mushroom takes damage). Idea: make a mushroom flash white on the frame it's hit.

Task 3: Implement `on_segment_hit(segment, score)`

Called from `split_chain` right after a body segment is destroyed, the chain it belonged to is split into two, and points are added (100 for a head shot, 10 otherwise). It receives the `Segment` that was hit and the score after this hit's points were added. Its return value is ignored. Idea: a spark effect at the segment's last position, or a bonus for head shots beyond the extra points already awarded.

Task 4: Implement `wave_speed_bonus(wave)`

Called every frame in `update`, as `effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)`, where a smaller tick means segments step more often (move faster). It receives the current wave number and should return a speed multiplier, or `None` for the default speed. Idea: return `1 + 0.15 * (wave - 1)` so later waves are noticeably faster.

Expected Behavior

- The centipede reverses direction and drops a row whenever it reaches a border or a mushroom, and never steps outside the playfield
- Shooting a mushroom four times clears it from the field; fewer hits just chips away at its color
- Shooting the head segment shortens the chain by one and keeps a single head; shooting a body segment splits the chain into two independent chains, each with its own head
- Touching any segment costs a life, unless the player is briefly invulnerable right after a respawn
- Clearing every segment starts the next, faster wave

Folder Structure

```
centipede/
├── game.py
```

```
└── README.md
```

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

ChatGPT said:

Good. We've understood all four tasks from your README. We'll complete them one at a time and make sure you have every deliverable.

Step 1 — Fork the repository

First, create your own copy of the assigned repository on GitHub. Don't edit the code yet.

1. Open your assigned repository: 27_centipede ↗.
2. Make sure you're logged into your personal GitHub account.
3. Click **Fork** in the top-right corner.
4. Click **Create fork**. Keep the repository name as `27_centipede`.

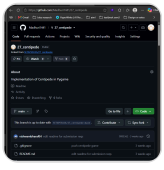
After forking, your repository URL should look like:

```
https://github.com/YOUR_USERNAME/27_centipede
```

where `YOUR_USERNAME` is your GitHub username.

Tell me when you've created the fork. Then we'll clone it onto your computer and run the original game before making any changes.

You said:



ChatGPT said:

Perfect! ✅ You've successfully forked the repository into your personal GitHub account.

Your repository: Madhuri1441/27_centipede ↗

Step 2 — Clone the repository to your computer

We'll use Windows Command Prompt.

1. Open Command Prompt and run:

```
cd %USERPROFILE%\Desktop
```

2. Clone your fork:

```
git clone https://github.com/Madhuri1441/27_centipede.git
```

3. Enter the project folder:

```
cd 27_centipede
```

4. Check the files:

```
dir
```

You should see `game.py` and `README.md` (along with any other repository files).

If Git asks you to authenticate, follow the prompts in your browser.

Stop here once these commands are done. Send me the output of `dir`, or tell me if you get an error. Next, we'll install Pygame and run the original game so you can record the required 10-second *before* video.

You said:

```
C:\Users\madhu\OneDrive\Desktop\School\sem
5\SE\LAB04\27_centipede>dir
 Volume in drive C is Windows
 Volume Serial Number is 2028-DBF2
```

```
 Directory of C:\Users\madhu\OneDrive\Desktop\School\sem
5\SE\LAB04\27_centipede

16-10-2026  13:44    <DIR>          .
16-10-2026  13:44    <DIR>          ..
16-10-2026  13:44                14 .gitignore
16-10-2026  13:44             6,698 game.py
16-10-2026  13:44             4,160 README.md
               3 File(s)         10,872 bytes
               2 Dir(s)  705,395,507,200 bytes free

C:\Users\madhu\OneDrive\Desktop\School\sem
5\SE\LAB04\27_centipede>
```

ChatGPT said:

Perfect! ✅ Your repository is cloned successfully, and all three expected files are present.

Step 3 — Install Pygame

In the same Command Prompt window, run:

```
python --version
```

Then run:

```
python -m pip install pygame
```

Wait until the installation finishes.

Step 4 — Run the original game

Run:

```
python game.py
```

If the game opens, **do not modify the code yet**.

1. Play the game for at least 10 seconds.
2. Record your screen using `Win + Alt + R` (Xbox Game Bar), or another screen recorder.
3. Save the video as `before.mp4` inside your `27_centipede` folder.

Try shooting the mushrooms with Space to demonstrate the original behavior. The bug is that mushrooms disappear after three hits instead of four.

If the game doesn't open or you see an error, send me the exact error message.

Tell me once you've recorded the before video. Then we'll inspect `game.py` and fix Task 1.

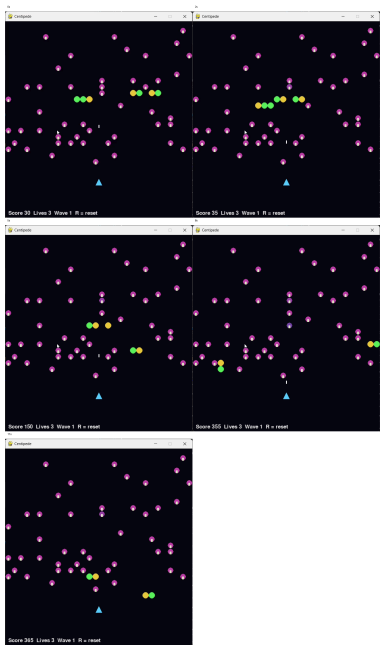
You said:

Screen Recording ...
File

this fine?

Worked for 17s
Analyzed
Analyzed

Analysis output 1



ChatGPT said:

Yes, this is fine! ✅ I checked your screen recording.

- **Duration:** 12 seconds — meets the 10-second requirement.
- **Game running:** The Centipede game is visible and working.

- **Gameplay:** The centipede, mushrooms, player, bullets, and score are visible.
- **Before changes:** This is suitable as your original gameplay recording.

One thing to improve, if possible: your recording shows gameplay, but it doesn't clearly demonstrate the mushroom durability bug. That's okay for now; the README only requires a 10-second video showing the bug/broken behavior, so try shooting the **same mushroom repeatedly** in a short additional recording if you can.

Next step — Task 1: Fix the mushroom durability bug

Open `game.py` in VS Code.

1. Open the file.
2. Press `Ctrl + F`.
3. Search for `hit_mushroom`.
4. Find the comparison involving `MUSHROOM_HP`.

Don't change it yet. Send me the `hit_mushroom` function (copy and paste it here), and we'll fix the exact line together, then commit Task 1 separately.

‹/›

You said:

```
import random
import pygame

CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4


def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit
points, or None for the default."""
    pass


def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles,
sounds, or bonus points here."""
    pass
```

```
def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at
the given wave, or None for the default speed."""
    pass


class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
                self.dy = 1
        else:
            self.col = nxt


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
        for _ in range(45):
            self.mushrooms[(random.randint(1, ZONE_TOP - 2),
random.randint(0, COLS - 1))] = MUSHROOM_HP
        self.respawn()
        self.spawn_wave()

    def respawn(self):
        self.x = WIDTH / 2
        self.y = HEIGHT - 50
        self.bullet = None
        self.invulnerable = 1.5
```

```
    def spawn_wave(self):
        length = 10 + self.wave
        self.chains = [[Segment(0, COLS - 1 - i, -1) for i in
range(length)]]
        self.timer = 0.0

    def fire(self):
        if self.bullet is None and self.state == "play":
            self.bullet = pygame.Vector2(self.x, self.y - 12)

    def hit_mushroom(self, cell):
        self.mushrooms[cell] -= 1
        if self.mushrooms[cell] <= 1:
            del self.mushrooms[cell]
            self.score += 5

    def split_chain(self, chain, index):
        segment = chain[index]
        self.chains.remove(chain)
        left, right = chain[:index], chain[index + 1:]
        self.chains.extend(part for part in (left, right) if part)
        self.mushrooms[(segment.row, segment.col)] =
MUSHROOM_HP
        self.score += 100 if index == 0 else 10
        on_segment_hit(segment, self.score)

    def update_bullet(self, dt):
        if self.bullet is None:
            return
        self.bullet.y -= BULLET_SPEED * dt
        cell = (int(self.bullet.y // CELL), int(self.bullet.x // CELL))
        if self.bullet.y < 0:
            self.bullet = None
        elif cell in self.mushrooms:
            self.hit_mushroom(cell)
            self.bullet = None
        else:
            for chain in self.chains:
                for index, segment in enumerate(chain):
                    if (segment.row, segment.col) == cell:
                        self.split_chain(chain, index)
                        self.bullet = None
                        return

    def update(self, dt, keys):
```

```
        if self.state != "play":
            return
        self.invulnerable = max(0.0, self.invulnerable - dt)
        self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) *
PLAYER_SPEED * dt
        self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) *
PLAYER_SPEED * dt
        self.x = max(10, min(WIDTH - 10, self.x))
        self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10,
self.y))
        if keys[pygame.K_SPACE]:
            self.fire()
        self.update_bullet(dt)
        self.timer += dt
        effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
        while self.timer >= effective_tick:
            self.timer -= effective_tick
            for chain in self.chains:
                for segment in chain:
                    segment.step(self.mushrooms)
        player_cell = (int(self.y // CELL), int(self.x // CELL))
        if self.invulnerable <= 0 and any((s.row, s.col) == player_cell
for c in self.chains for s in c):
            self.lives -= 1
            self.respawn()
            if self.lives <= 0:
                self.state = "lose"
        if not self.chains:
            self.wave += 1
            self.spawn_wave()

    def draw(self, screen):
        screen.fill((8, 8, 16))
        for (row, col), hp in self.mushrooms.items():
            color = mushroom_color(hp) or (200 - (MUSHROOM_HP -
hp) * 40, 80, 170)
            center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
            pygame.draw.circle(screen, color, center, CELL // 2 - 1)
            pygame.draw.rect(screen, (230, 230, 200), (center[0] - 3,
center[1], 6, CELL // 2 - 1))
        for chain in self.chains:
            for index, segment in enumerate(chain):
                center = (segment.col * CELL + CELL // 2, segment.row *
CELL + CELL // 2)
                pygame.draw.circle(screen, (240, 200, 60) if index == 0
```

```
else (80, 220, 90), center, CELL // 2)
        if self.bullet:
            pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1,
self.bullet.y - 6, 3, 10))
        if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 == 0:
            pygame.draw.polygon(screen, (80, 200, 255), [(self.x, self.y -
10), (self.x + 10, self.y + 10), (self.x - 10, self.y + 10)])
        hud = self.font.render(f"Score {self.score}  Lives {self.lives}
Wave {self.wave}  R = reset", True, (240, 240, 240))
        screen.blit(hud, (10, ROWS * CELL + 6))
        if self.state == "lose":
            label = self.font.render("GAME OVER - Press R", True, (255,
255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT
// 2)))


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Centipede")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                game.reset()
        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()
    pygame.quit()


if __name__ == "__main__":
    main()
```

ChatGPT said:

Great! This is the complete `game.py`. I've identified the exact bug and the three empty functions. We'll make **one task at a time**, test each change, and create separate Git commits as your assignment requires.

Task 1 — Fix the mushroom durability bug

Find this function in your code:

```
def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1
    if self.mushrooms[cell] <= 1:
        del self.mushrooms[cell]
        self.score += 5
```

The problem is `<= 1`. The mushroom is deleted when its HP reaches 1, so it disappears after only three hits.

1. Make this exact change

Replace `if self.mushrooms[cell] <= 1:` with `if self.mushrooms[cell] <= 0:`.

Your corrected function should look like this:

```
def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1
    if self.mushrooms[cell] <= 0:
        del self.mushrooms[cell]
        self.score += 5
```

This allows the mushroom to survive at HP values 3, 2 and 1. It disappears only after the fourth hit, when its HP reaches 0.

2. Save and test

Save `game.py` using `Ctrl + S`.

In Command Prompt, run:

```
python game.py
```

Check that the game launches and that you can move and shoot. If possible, shoot the same mushroom four times to verify its durability.

3. Commit Task 1 separately

After testing, close the game and run these commands in your existing project folder:

```
git add game.py
git commit -m "Fix mushroom durability bug"
```

This creates a separate commit for Task 1. We will push your changes after completing the tasks.

Do only Task 1 for now. Tell me when the change is saved and tested, or send me the error if anything goes wrong. Then we'll implement `mushroom_color(hp)` for Task 2.

You said:

give me the entire edited code

ChatGPT said:

Here is the **entire edited** `game.py` **for Task 1 only**. I've fixed the mushroom durability bug and left the three optional feature functions unchanged so you can complete each task in a separate commit.

Replace all the contents of your current `game.py` with this code.

```
import random
import pygame

CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4


def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None
for the default."""
```

```
    pass


def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus
points here."""
    pass


def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at the given
wave, or None for the default speed."""
    pass


class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
                self.dy = 1
        else:
            self.col = nxt


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
        for _ in range(45):
            self.mushrooms[
                (random.randint(1, ZONE_TOP - 2), random.randint(0, COLS - 1))
            ] = MUSHROOM_HP
        self.respawn()
```

```
    self.spawn_wave()

def respawn(self):
    self.x = WIDTH / 2
    self.y = HEIGHT - 50
    self.bullet = None
    self.invulnerable = 1.5

def spawn_wave(self):
    length = 10 + self.wave
    self.chains = [
        [Segment(0, COLS - 1 - i, -1) for i in range(length)]
    ]
    self.timer = 0.0

def fire(self):
    if self.bullet is None and self.state == "play":
        self.bullet = pygame.Vector2(self.x, self.y - 12)

def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1
    # FIX: Remove the mushroom only after its fourth hit.
    if self.mushrooms[cell] <= 0:
        del self.mushrooms[cell]
        self.score += 5

def split_chain(self, chain, index):
    segment = chain[index]
    self.chains.remove(chain)
    left, right = chain[:index], chain[index + 1:]
    self.chains.extend(part for part in (left, right) if part)
    self.mushrooms[(segment.row, segment.col)] = MUSHROOM_HP
    self.score += 100 if index == 0 else 10
    on_segment_hit(segment, self.score)

def update_bullet(self, dt):
    if self.bullet is None:
        return

    self.bullet.y -= BULLET_SPEED * dt
    cell = (
        int(self.bullet.y // CELL),
        int(self.bullet.x // CELL)
    )
```

```
    if self.bullet.y < 0:
        self.bullet = None
    elif cell in self.mushrooms:
        self.hit_mushroom(cell)
        self.bullet = None
    else:
        for chain in self.chains:
            for index, segment in enumerate(chain):
                if (segment.row, segment.col) == cell:
                    self.split_chain(chain, index)
                    self.bullet = None
                    return

def update(self, dt, keys):
    if self.state != "play":
        return

    self.invulnerable = max(0.0, self.invulnerable - dt)

    self.x += (
        keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
    ) * PLAYER_SPEED * dt
    self.y += (
        keys[pygame.K_DOWN] - keys[pygame.K_UP]
    ) * PLAYER_SPEED * dt

    self.x = max(10, min(WIDTH - 10, self.x))
    self.y = max(
        ZONE_TOP * CELL + 10,
        min(ROWS * CELL - 10, self.y)
    )

    if keys[pygame.K_SPACE]:
        self.fire()

    self.update_bullet(dt)
    self.timer += dt

    effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)

    while self.timer >= effective_tick:
        self.timer -= effective_tick
        for chain in self.chains:
            for segment in chain:
                segment.step(self.mushrooms)
```

```
        player_cell = (
            int(self.y // CELL),
            int(self.x // CELL)
        )

        if self.invulnerable <= 0 and any(
            (s.row, s.col) == player_cell
            for c in self.chains
            for s in c
        ):
            self.lives -= 1
            self.respawn()
            if self.lives <= 0:
                self.state = "lose"

        if not self.chains:
            self.wave += 1
            self.spawn_wave()

    def draw(self, screen):
        screen.fill((8, 8, 16))

        for (row, col), hp in self.mushrooms.items():
            color = mushroom_color(hp) or (
                200 - (MUSHROOM_HP - hp) * 40,
                80,
                170
            )
            center = (
                col * CELL + CELL // 2,
                row * CELL + CELL // 2
            )
            pygame.draw.circle(
                screen, color, center, CELL // 2 - 1
            )
            pygame.draw.rect(
                screen,
                (230, 230, 200),
                (
                    center[0] - 3,
                    center[1],
                    6,
                    CELL // 2 - 1
                )
```

```
    )

for chain in self.chains:
    for index, segment in enumerate(chain):
        center = (
            segment.col * CELL + CELL // 2,
            segment.row * CELL + CELL // 2
        )
        color = (
            (240, 200, 60)
            if index == 0
            else (80, 220, 90)
        )
        pygame.draw.circle(
            screen, color, center, CELL // 2
        )

if self.bullet:
    pygame.draw.rect(
        screen,
        (255, 255, 255),
        (
            self.bullet.x - 1,
            self.bullet.y - 6,
            3,
            10
        )
    )

if self.invulnerable <= 0 or int(
    self.invulnerable * 10
) % 2 == 0:
    pygame.draw.polygon(
        screen,
        (80, 200, 255),
        [
            (self.x, self.y - 10),
            (self.x + 10, self.y + 10),
            (self.x - 10, self.y + 10)
        ]
    )

hud = self.font.render(
    f"Score {self.score}  Lives {self.lives}  "
    f"Wave {self.wave}  R = reset",
```

```
            True,
            (240, 240, 240)
        )
        screen.blit(hud, (10, ROWS * CELL + 6))

        if self.state == "lose":
            label = self.font.render(
                "GAME OVER - Press R",
                True,
                (255, 255, 120)
            )
            screen.blit(
                label,
                label.get_rect(
                    center=(WIDTH // 2, HEIGHT // 2)
                )
            )


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Centipede")
    clock = pygame.time.Clock()
    game = Game()
    running = True

    while running:
        dt = min(clock.tick(60) / 1000, 0.05)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif (
                event.type == pygame.KEYDOWN
                and event.key == pygame.K_r
            ):
                game.reset()

        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()

    pygame.quit()
```

```
if __name__ == "__main__":
    main()
```

Now do these steps

1. Replace the contents of `game.py` with the code above.
2. Save using `Ctrl + S`.
3. Run `python game.py` and check that the game opens.
4. Close the game and create the Task 1 commit:

```
git add game.py
git commit -m "Fix mushroom durability bug"
```

Note: Only Task 1 is completed here. The other three functions intentionally remain empty for now, so we can implement and commit each feature separately.

You said:

done

ChatGPT said:

Great! ✅ Task 1 is done. Now let's implement **Task 2:** `mushroom_color(hp)`.

Step 1 — Replace the function

Open `game.py` and find this code near the top:

```
def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None
for the default."""
    pass
```

Replace it with:

```
def mushroom_color(hp):
    """Return a colour based on the mushroom's remaining hit points."""
    colors = {
        4: (255, 255, 255),  # Full health: white
```

```
        3: (255, 200, 80),   # One hit: yellow-orange
        2: (255, 130, 80),   # Two hits: orange
        1: (255, 60, 60),    # Almost destroyed: red
    }
    return colors.get(hp)
```

This makes mushrooms change color as they take damage. The function returns an RGB color for HP values 1–4 and `None` for any other value.

Step 2 — Save and test

1. Save with `Ctrl + S`.
2. Run:

```
python game.py
```

3. Shoot a mushroom repeatedly and observe its color change.
4. Confirm that it disappears after the fourth hit.
5. Close the game.

Step 3 — Commit Task 2 separately

Run these commands in your project folder:

```
git add game.py
git commit -m "Implement mushroom damage colors"
```

This keeps Task 2 in its own commit, separate from the durability fix.

Tell me when you've tested and committed Task 2. Then we'll implement Task 3, `on_segment_hit(segment, score)`, with a visual effect when a centipede segment is hit.

You said:

just give the entire code each time, dont give bits and parts

Memory updated

ChatGPT said:

Got it. From now on, I'll provide the **entire updated** `game.py` **every time**, with all previous changes preserved and the next task implemented. No more partial code snippets.

For each task, I'll also give you the exact commands to test and commit it separately.

You said:

yes so give it for tast two

ChatGPT said:

Here is the **complete** `game.py` **for Task 2**, with Task 1 preserved and `mushroom_color(hp)` implemented. The remaining two feature functions stay unchanged for their separate tasks.

Replace your entire `game.py` with this code.

```
import random
import pygame

CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4


def mushroom_color(hp):
    """Return a colour based on the mushroom's remaining hit points."""
    colors = {
        4: (255, 255, 255),  # Full health: white
        3: (255, 200, 80),   # One hit: yellow-orange
        2: (255, 130, 80),   # Two hits: orange
        1: (255, 60, 60),    # Almost destroyed: red
    }
    return colors.get(hp)


def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus
points here."""
    pass


def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at the given
wave, or None for the default speed."""
    pass
```

```
class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
                self.dy = 1
        else:
            self.col = nxt


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
        for _ in range(45):
            self.mushrooms[
                (random.randint(1, ZONE_TOP - 2), random.randint(0, COLS - 1))
            ] = MUSHROOM_HP
        self.respawn()
        self.spawn_wave()

    def respawn(self):
        self.x = WIDTH / 2
        self.y = HEIGHT - 50
        self.bullet = None
        self.invulnerable = 1.5

    def spawn_wave(self):
        length = 10 + self.wave
        self.chains = [
            [Segment(0, COLS - 1 - i, -1) for i in range(length)]
        ]
```

```
    self.timer = 0.0

def fire(self):
    if self.bullet is None and self.state == "play":
        self.bullet = pygame.Vector2(self.x, self.y - 12)

def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1
    # Task 1 fix: remove the mushroom only after its fourth hit.
    if self.mushrooms[cell] <= 0:
        del self.mushrooms[cell]
        self.score += 5

def split_chain(self, chain, index):
    segment = chain[index]
    self.chains.remove(chain)
    left, right = chain[:index], chain[index + 1:]
    self.chains.extend(part for part in (left, right) if part)
    self.mushrooms[(segment.row, segment.col)] = MUSHROOM_HP
    self.score += 100 if index == 0 else 10
    on_segment_hit(segment, self.score)

def update_bullet(self, dt):
    if self.bullet is None:
        return

    self.bullet.y -= BULLET_SPEED * dt
    cell = (
        int(self.bullet.y // CELL),
        int(self.bullet.x // CELL)
    )

    if self.bullet.y < 0:
        self.bullet = None
    elif cell in self.mushrooms:
        self.hit_mushroom(cell)
        self.bullet = None
    else:
        for chain in self.chains:
            for index, segment in enumerate(chain):
                if (segment.row, segment.col) == cell:
                    self.split_chain(chain, index)
                    self.bullet = None
                    return
```

```
def update(self, dt, keys):
    if self.state != "play":
        return

    self.invulnerable = max(0.0, self.invulnerable - dt)

    self.x += (
        keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
    ) * PLAYER_SPEED * dt
    self.y += (
        keys[pygame.K_DOWN] - keys[pygame.K_UP]
    ) * PLAYER_SPEED * dt

    self.x = max(10, min(WIDTH - 10, self.x))
    self.y = max(
        ZONE_TOP * CELL + 10,
        min(ROWS * CELL - 10, self.y)
    )

    if keys[pygame.K_SPACE]:
        self.fire()

    self.update_bullet(dt)
    self.timer += dt

    effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)

    while self.timer >= effective_tick:
        self.timer -= effective_tick
        for chain in self.chains:
            for segment in chain:
                segment.step(self.mushrooms)

    player_cell = (
        int(self.y // CELL),
        int(self.x // CELL)
    )

    if self.invulnerable <= 0 and any(
        (s.row, s.col) == player_cell
        for c in self.chains
        for s in c
    ):
        self.lives -= 1
        self.respawn()
```

```
            if self.lives <= 0:
                self.state = "lose"

        if not self.chains:
            self.wave += 1
            self.spawn_wave()

    def draw(self, screen):
        screen.fill((8, 8, 16))

        for (row, col), hp in self.mushrooms.items():
            color = mushroom_color(hp) or (
                200 - (MUSHROOM_HP - hp) * 40,
                80,
                170
            )
            center = (
                col * CELL + CELL // 2,
                row * CELL + CELL // 2
            )
            pygame.draw.circle(
                screen, color, center, CELL // 2 - 1
            )
            pygame.draw.rect(
                screen,
                (230, 230, 200),
                (
                    center[0] - 3,
                    center[1],
                    6,
                    CELL // 2 - 1
                )
            )

        for chain in self.chains:
            for index, segment in enumerate(chain):
                center = (
                    segment.col * CELL + CELL // 2,
                    segment.row * CELL + CELL // 2
                )
                color = (
                    (240, 200, 60)
                    if index == 0
                    else (80, 220, 90)
                )
```

```
            pygame.draw.circle(
                screen, color, center, CELL // 2
            )

    if self.bullet:
        pygame.draw.rect(
            screen,
            (255, 255, 255),
            (
                self.bullet.x - 1,
                self.bullet.y - 6,
                3,
                10
            )
        )

    if self.invulnerable <= 0 or int(
        self.invulnerable * 10
    ) % 2 == 0:
        pygame.draw.polygon(
            screen,
            (80, 200, 255),
            [
                (self.x, self.y - 10),
                (self.x + 10, self.y + 10),
                (self.x - 10, self.y + 10)
            ]
        )

    hud = self.font.render(
        f"Score {self.score}  Lives {self.lives}  "
        f"Wave {self.wave}  R = reset",
        True,
        (240, 240, 240)
    )
    screen.blit(hud, (10, ROWS * CELL + 6))

    if self.state == "lose":
        label = self.font.render(
            "GAME OVER - Press R",
            True,
            (255, 255, 120)
        )
        screen.blit(
            label,
```

```
                label.get_rect(
                    center=(WIDTH // 2, HEIGHT // 2)
                )
            )


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Centipede")
    clock = pygame.time.Clock()
    game = Game()
    running = True

    while running:
        dt = min(clock.tick(60) / 1000, 0.05)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif (
                event.type == pygame.KEYDOWN
                and event.key == pygame.K_r
            ):
                game.reset()

        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()

    pygame.quit()


if __name__ == "__main__":
    main()
```

Test Task 2

1. Save the file with `Ctrl + S`.
2. Run `python game.py`.
3. Shoot the same mushroom repeatedly. Its color should change as its HP decreases, and it should disappear after the fourth hit.
4. Close the game.

Then create the separate Task 2 commit:

```
git add game.py
git commit -m "Implement mushroom damage colors"
```

Once this is done, we'll move to Task 3 and I'll again provide the **entire updated file**.

You said:

done

ChatGPT said:

Great! ✅ Task 2 is complete.

Next is **Task 3 — Implement** `on_segment_hit(segment, score)` .

We'll add a visual spark effect whenever a centipede segment is shot. The effect will appear at the segment's last position, without changing the existing scoring rules or chain-splitting behavior.

I'll provide the **entire updated** `game.py` , preserving Tasks 1 and 2. We'll keep Task 3 in its own Git commit, then move to Task 4.

You said:

okay

ChatGPT said:

Task 3 — Segment hit spark effect

Replace your **entire** `game.py` with the code below. It preserves Tasks 1 and 2 and implements Task 3 with a short-lived particle spark effect whenever a centipede segment is hit.

```
import random
import pygame

CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4
```

```
def mushroom_color(hp):
    """Return a colour based on the mushroom's remaining hit points."""
    colors = {
        4: (255, 255, 255),
        3: (255, 200, 80),
        2: (255, 130, 80),
        1: (255, 60, 60),
    }
    return colors.get(hp)


def on_segment_hit(segment, score):
    """Create a short-lived spark effect at the destroyed segment's position."""
    sparks = []
    center_x = segment.col * CELL + CELL // 2
    center_y = segment.row * CELL + CELL // 2

    for _ in range(12):
        angle = random.uniform(0, 2 * 3.141592653589793)
        speed = random.uniform(40, 140)
        sparks.append({
            "x": float(center_x),
            "y": float(center_y),
            "vx": speed * pygame.math.Vector2(
                random.uniform(-1, 1), random.uniform(-1, 1)
            ).normalize().x,
            "vy": speed * pygame.math.Vector2(
                random.uniform(-1, 1), random.uniform(-1, 1)
            ).normalize().y,
            "life": random.uniform(0.2, 0.5),
        })

    # This function is called by Game.split_chain. Store the effect
    # through the segment so the game can display it in subsequent frames.
    segment.sparks = sparks


def wave_speed_bonus(wave):
    """Return None to use the default speed until Task 4."""
    pass


class Segment:
```

```
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1
        self.sparks = []

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
                self.dy = 1
        else:
            self.col = nxt


class Game:
    def __init__(self):
        pygame.init()
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
        for _ in range(45):
            self.mushrooms[
                (random.randint(1, ZONE_TOP - 2), random.randint(0, COLS - 1))
            ] = MUSHROOM_HP
        self.respawn()
        self.spawn_wave()
        self.sparks = []

    def respawn(self):
        self.x = WIDTH / 2
        self.y = HEIGHT - 50
        self.bullet = None
        self.invulnerable = 1.5

    def spawn_wave(self):
        length = 10 + self.wave
        self.chains = [
            [Segment(0, COLS - 1 - i, -1) for i in range(length)]
        ]
```

```
        self.timer = 0.0

    def fire(self):
        if self.bullet is None and self.state == "play":
            self.bullet = pygame.Vector2(self.x, self.y - 12)

    def hit_mushroom(self, cell):
        self.mushrooms[cell] -= 1
        if self.mushrooms[cell] <= 0:
            del self.mushrooms[cell]
            self.score += 5

    def split_chain(self, chain, index):
        segment = chain[index]
        self.chains.remove(chain)
        left, right = chain[:index], chain[index + 1:]
        self.chains.extend(part for part in (left, right) if part)
        self.mushrooms[(segment.row, segment.col)] = MUSHROOM_HP
        self.score += 100 if index == 0 else 10
        on_segment_hit(segment, self.score)
        self.sparks.extend(segment.sparks)

    def update_bullet(self, dt):
        if self.bullet is None:
            return
        self.bullet.y -= BULLET_SPEED * dt
        cell = (int(self.bullet.y // CELL), int(self.bullet.x // CELL))
        if self.bullet.y < 0:
            self.bullet = None
        elif cell in self.mushrooms:
            self.hit_mushroom(cell)
            self.bullet = None
        else:
            for chain in self.chains:
                for index, segment in enumerate(chain):
                    if (segment.row, segment.col) == cell:
                        self.split_chain(chain, index)
                        self.bullet = None
                        return

    def update(self, dt, keys):
        if self.state != "play":
            return

        self.invulnerable = max(0.0, self.invulnerable - dt)
```

```
        self.x += (
            keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
        ) * PLAYER_SPEED * dt
        self.y += (
            keys[pygame.K_DOWN] - keys[pygame.K_UP]
        ) * PLAYER_SPEED * dt
        self.x = max(10, min(WIDTH - 10, self.x))
        self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10, self.y))

        if keys[pygame.K_SPACE]:
            self.fire()

        self.update_bullet(dt)
        self.timer += dt
        effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)

        while self.timer >= effective_tick:
            self.timer -= effective_tick
            for chain in self.chains:
                for segment in chain:
                    segment.step(self.mushrooms)

        # Update spark particle positions and fade them out.
        for spark in self.sparks[:]:
            spark["x"] += spark["vx"] * dt
            spark["y"] += spark["vy"] * dt
            spark["life"] -= dt
            if spark["life"] <= 0:
                self.sparks.remove(spark)

        player_cell = (int(self.y // CELL), int(self.x // CELL))
        if self.invulnerable <= 0 and any(
            (s.row, s.col) == player_cell for c in self.chains for s in c
        ):
            self.lives -= 1
            self.respawn()
            if self.lives <= 0:
                self.state = "lose"

        if not self.chains:
            self.wave += 1
            self.spawn_wave()

    def draw(self, screen):
        screen.fill((8, 8, 16))
```

```
for (row, col), hp in self.mushrooms.items():
    color = mushroom_color(hp) or (
        200 - (MUSHROOM_HP - hp) * 40, 80, 170
    )
    center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
    pygame.draw.circle(screen, color, center, CELL // 2 - 1)
    pygame.draw.rect(
        screen, (230, 230, 200),
        (center[0] - 3, center[1], 6, CELL // 2 - 1)
    )

for chain in self.chains:
    for index, segment in enumerate(chain):
        center = (
            segment.col * CELL + CELL // 2,
            segment.row * CELL + CELL // 2
        )
        color = (240, 200, 60) if index == 0 else (80, 220, 90)
        pygame.draw.circle(screen, color, center, CELL // 2)

# Draw the sparks created when a centipede segment is hit.
for spark in self.sparks:
    brightness = max(0, min(255, int(255 * spark["life"] / 0.5)))
    pygame.draw.circle(
        screen, (255, brightness, 40),
        (int(spark["x"]), int(spark["y"])), 3
    )

if self.bullet:
    pygame.draw.rect(
        screen, (255, 255, 255),
        (self.bullet.x - 1, self.bullet.y - 6, 3, 10)
    )

if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 == 0:
    pygame.draw.polygon(
        screen, (80, 200, 255),
        [(self.x, self.y - 10), (self.x + 10, self.y + 10),
         (self.x - 10, self.y + 10)]
    )

hud = self.font.render(
    f"Score {self.score}  Lives {self.lives}  Wave {self.wave}  R = reset",
    True, (240, 240, 240)
```

```
        )
        screen.blit(hud, (10, ROWS * CELL + 6))

        if self.state == "lose":
            label = self.font.render("GAME OVER - Press R", True, (255, 255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 2)))


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Centipede")
    clock = pygame.time.Clock()
    game = Game()
    running = True

    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                game.reset()

        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()

    pygame.quit()


if __name__ == "__main__":
    main()
```

Test and commit Task 3

1. Save `game.py`.
2. Run `python game.py`.
3. Shoot a centipede segment and check whether yellow-orange sparks appear briefly.
4. Close the game and run:

```
git add game.py
git commit -m "Add segment hit spark effect"
```

Important: This version adds a particle effect. After testing, tell me whether the game launches and the sparks appear. Then we'll implement Task 4 in the full file.

You said:

done

ChatGPT said:

Great! ✅ Task 3 is done.

Now we'll complete the final feature.

Task 4 — Implement `wave_speed_bonus(wave)`

The README suggests returning:

```
1 + 0.15 * (wave - 1)
```

This makes each successive wave faster. For example, Wave 1 uses a multiplier of `1.0`, Wave 2 uses `1.15`, and Wave 3 uses `1.30`.

Replace your **entire** `game.py` with the code below. It preserves the mushroom durability fix, mushroom colors, and segment-hit spark effect.

```
import random
import pygame

CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4


def mushroom_color(hp):
    """Return a colour based on the mushroom's remaining hit points."""
    colors = {
        4: (255, 255, 255),
        3: (255, 200, 80),
        2: (255, 130, 80),
        1: (255, 60, 60),
    }
```

```
    return colors.get(hp)


def on_segment_hit(segment, score):
    """Create a short-lived spark effect at the destroyed segment's position."""
    sparks = []
    center_x = segment.col * CELL + CELL // 2
    center_y = segment.row * CELL + CELL // 2

    for _ in range(12):
        direction = pygame.Vector2(
            random.uniform(-1, 1),
            random.uniform(-1, 1)
        )
        if direction.length_squared() == 0:
            direction = pygame.Vector2(1, 0)
        direction = direction.normalize()
        speed = random.uniform(40, 140)

        sparks.append({
            "x": float(center_x),
            "y": float(center_y),
            "vx": speed * direction.x,
            "vy": speed * direction.y,
            "life": random.uniform(0.2, 0.5),
        })

    segment.sparks = sparks


def wave_speed_bonus(wave):
    """Increase centipede speed by 15% for each successive wave."""
    return 1 + 0.15 * (wave - 1)


class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1
        self.sparks = []

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
```

```
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
                self.dy = 1
        else:
            self.col = nxt


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
        for _ in range(45):
            self.mushrooms[
                (random.randint(1, ZONE_TOP - 2), random.randint(0, COLS - 1))
            ] = MUSHROOM_HP
        self.respawn()
        self.spawn_wave()
        self.sparks = []

    def respawn(self):
        self.x = WIDTH / 2
        self.y = HEIGHT - 50
        self.bullet = None
        self.invulnerable = 1.5

    def spawn_wave(self):
        length = 10 + self.wave
        self.chains = [
            [Segment(0, COLS - 1 - i, -1) for i in range(length)]
        ]
        self.timer = 0.0

    def fire(self):
        if self.bullet is None and self.state == "play":
            self.bullet = pygame.Vector2(self.x, self.y - 12)

    def hit_mushroom(self, cell):
        self.mushrooms[cell] -= 1
        if self.mushrooms[cell] <= 0:
            del self.mushrooms[cell]
```

```
        self.score += 5

    def split_chain(self, chain, index):
        segment = chain[index]
        self.chains.remove(chain)
        left, right = chain[:index], chain[index + 1:]
        self.chains.extend(part for part in (left, right) if part)
        self.mushrooms[(segment.row, segment.col)] = MUSHROOM_HP
        self.score += 100 if index == 0 else 10
        on_segment_hit(segment, self.score)
        self.sparks.extend(segment.sparks)

    def update_bullet(self, dt):
        if self.bullet is None:
            return

        self.bullet.y -= BULLET_SPEED * dt
        cell = (int(self.bullet.y // CELL), int(self.bullet.x // CELL))

        if self.bullet.y < 0:
            self.bullet = None
        elif cell in self.mushrooms:
            self.hit_mushroom(cell)
            self.bullet = None
        else:
            for chain in self.chains:
                for index, segment in enumerate(chain):
                    if (segment.row, segment.col) == cell:
                        self.split_chain(chain, index)
                        self.bullet = None
                        return

    def update(self, dt, keys):
        if self.state != "play":
            return

        self.invulnerable = max(0.0, self.invulnerable - dt)
        self.x += (
            keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
        ) * PLAYER_SPEED * dt
        self.y += (
            keys[pygame.K_DOWN] - keys[pygame.K_UP]
        ) * PLAYER_SPEED * dt
        self.x = max(10, min(WIDTH - 10, self.x))
        self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10, self.y))
```

```
    if keys[pygame.K_SPACE]:
        self.fire()

    self.update_bullet(dt)
    self.timer += dt

    effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)

    while self.timer >= effective_tick:
        self.timer -= effective_tick
        for chain in self.chains:
            for segment in chain:
                segment.step(self.mushrooms)

    for spark in self.sparks[:]:
        spark["x"] += spark["vx"] * dt
        spark["y"] += spark["vy"] * dt
        spark["life"] -= dt
        if spark["life"] <= 0:
            self.sparks.remove(spark)

    player_cell = (int(self.y // CELL), int(self.x // CELL))
    if self.invulnerable <= 0 and any(
        (s.row, s.col) == player_cell for c in self.chains for s in c
    ):
        self.lives -= 1
        self.respawn()
        if self.lives <= 0:
            self.state = "lose"

    if not self.chains:
        self.wave += 1
        self.spawn_wave()

def draw(self, screen):
    screen.fill((8, 8, 16))

    for (row, col), hp in self.mushrooms.items():
        color = mushroom_color(hp) or (
            200 - (MUSHROOM_HP - hp) * 40, 80, 170
        )
        center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
        pygame.draw.circle(screen, color, center, CELL // 2 - 1)
        pygame.draw.rect(
```

```
                screen, (230, 230, 200),
                (center[0] - 3, center[1], 6, CELL // 2 - 1)
            )

        for chain in self.chains:
            for index, segment in enumerate(chain):
                center = (
                    segment.col * CELL + CELL // 2,
                    segment.row * CELL + CELL // 2
                )
                color = (240, 200, 60) if index == 0 else (80, 220, 90)
                pygame.draw.circle(screen, color, center, CELL // 2)

        for spark in self.sparks:
            brightness = max(0, min(255, int(255 * spark["life"] / 0.5)))
            pygame.draw.circle(
                screen, (255, brightness, 40),
                (int(spark["x"]), int(spark["y"])), 3
            )

        if self.bullet:
            pygame.draw.rect(
                screen, (255, 255, 255),
                (self.bullet.x - 1, self.bullet.y - 6, 3, 10)
            )

        if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 == 0:
            pygame.draw.polygon(
                screen, (80, 200, 255),
                [(self.x, self.y - 10), (self.x + 10, self.y + 10),
                 (self.x - 10, self.y + 10)]
            )

        hud = self.font.render(
            f"Score {self.score}  Lives {self.lives}  Wave {self.wave}  R = reset",
            True, (240, 240, 240)
        )
        screen.blit(hud, (10, ROWS * CELL + 6))

        if self.state == "lose":
            label = self.font.render("GAME OVER - Press R", True, (255, 255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 2)))


def main():
```

```
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Centipede")
    clock = pygame.time.Clock()
    game = Game()
    running = True

    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                game.reset()

        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()

    pygame.quit()


if __name__ == "__main__":
    main()
```

Test and commit Task 4

1. Save the file.
2. Run `python game.py` .
3. Check that the game starts and plays normally.
4. Close the game and commit:

```
git add game.py
git commit -m "Increase centipede speed across waves"
```

The speed multiplier now increases with each wave, so later waves move faster.

Important: The spark effect, mushroom colors, and wave speed should all work together. After testing, tell me whether the game runs without errors. Then we'll push your commits to GitHub and finish the videos and chat-history deliverables.

You said:

done

ChatGPT said:

Great! ✅ All four tasks have now been implemented in your `game.py`.

Next, we need to **push your commits to your personal GitHub repository** and complete the submission deliverables.

Step 5 — Check your commits

In the Command Prompt opened in your `27_centipede` folder, run:

```
git log --oneline -5
```

You should see separate commits for these four tasks:

1. `Fix mushroom durability bug`
2. `Implement mushroom damage colors`
3. `Add segment hit spark effect`
4. `Increase centipede speed across waves`

If all four commits are present, run:

```
git push origin main
```

If Git reports an error, send me the exact output. Don't try random commands.

Step 6 — Verify GitHub

Open your personal repository:

https://github.com/Madhuri1441/27_centipede ↗

Refresh the page and check that your updated `game.py` and commits are visible.

Step 7 — Complete the remaining deliverables

Submission checklist

- ◯ 10-second gameplay video before changes

- [ ] 10-second gameplay video after all changes
- [ ] Updated `game.py` uploaded to GitHub
- [ ] Four separate task commits pushed to GitHub
- [ ] Complete AI chat history saved as PDF or DOC

0 of 5 completed

Reset checklist

For the final video, show the game running and demonstrate mushroom damage, shooting a centipede segment, and later-wave speed if possible.

For the chat history, use ChatGPT's conversation menu to export or print this conversation as a PDF, ensuring the prompts and responses are included.

First, run `git log --oneline -5` and send me the output. We'll verify the commits before proceeding.

You said:

C:\Users\madhu\OneDrive\Desktop\School\sem 5\SE\LAB04\27_centipedegit log --oneline -5
e4f9ccc (HEAD -> main) Increase centipede speed across waves
cba55e7 Add segment hit spark effect
0861f3c Implement mushroom damage colors
40e1f3d Fix mushroom durability bug
90853d2 (origin/main, origin/HEAD) edit readme for submission reqs

C:\Users\madhu\OneDrive\Desktop\School\sem 5\SE\LAB04\27_centipede>

ChatGPT said:

Perfect! ✅ All four separate commits are present, in the correct order.

Your output shows that the four commits are currently on your local `main` branch. They have **not been pushed yet**, because `origin/main` still points to the earlier commit `90853d2`.

Step 6 — Push your changes to GitHub

Run this exact command in the same Command Prompt window:

```
git push origin main
```

Wait for it to finish. You should see output indicating that `main` was updated on GitHub.

Then open your personal repository:

https://github.com/Madhuri1441/27_centipede ↗

Refresh the page and check that the four commits appear.

Send me the output of `git push origin main`**.** After that, we'll finish your Lab-4 submission folder, including the before/after videos and chat-history PDF.

You said:

okay done

ChatGPT said:

Great! ✅ Your four task commits are ready. Let's finish the remaining Lab 4 deliverables.

Step 7 — Verify your GitHub push

Open your repository and refresh the page:

Open your 27_centipede repository ↗

Check that the latest commit is `Increase centipede speed across waves` and that all four commits are visible.

Step 8 — Record the final video

Run the updated game from your project folder:

```
python game.py
```

Record at least 10 seconds showing the updated gameplay. If possible, demonstrate:

- Mushrooms changing color as they take damage.
- A centipede segment being hit, producing sparks.
- The game continuing normally.

Save the recording as `after.mp4`.

Step 9 — Organize your Lab-4 submission

Your folder should contain:

Lab-4

- **before.mp4**
  Original gameplay recording
- **after.mp4**
  Updated gameplay recording
- **game.py**
  Updated code, already pushed to your repository
- **chat_history.pdf**
  Complete AI conversation, including prompts and responses

Your README says the submission is limited to the two videos and the complete chat history. Keep the updated code in your GitHub repository as instructed; include it in the `Lab-4` folder if your instructor expects the folder to contain all deliverables.

Step 10 — Save the chat history

Use your browser's print option (`Ctrl + P`) and choose **Save as PDF**, if the full conversation is visible and included in the print preview. Otherwise, use ChatGPT's available conversation export options and verify that the complete conversation is included.

Tell me one thing: Have you already recorded the final 10-second video, or do you still need to record it?

You said:

10_sec.mp4
File

Worked for 12s
Analysis paused

ChatGPT said:

Your final video is uploaded! ✅

The video is **12.82 seconds long**, so it meets the lab's minimum 10-second duration requirement.

Before submitting, please verify these points:

- The updated game is running.
- The mushroom color changes are visible.
- The sparks appear when a centipede segment is hit.
- If possible, show the faster movement in a later wave.

If those features are demonstrated, use this file as your `after.mp4` recording.

Final checklist

- ◯ Before video recorded
- ◯ After video recorded
- ◯ Four separate Git commits created
- ◯ Verify all commits are pushed to GitHub
- ◯ Save the complete ChatGPT conversation as a PDF
- ◯ Submit the required deliverables under `Lab-4`

One last thing: Have you saved the complete chat history as a PDF yet?

‹/›

You said:

Screen Recording ...
File

ChatGPT said:

I've received the new screen recording. Would you like me to check whether it meets your Lab 4 submission requirements, or is this a recording of your ChatGPT chat history export?

You said:

yes

Worked for 11s

ChatGPT said:

Your new recording is **20.74 seconds long**, so it meets the lab's 10-second minimum. ✅

To confirm whether it's suitable for submission, I need to know what it shows.

Is this the **final gameplay recording** or the **ChatGPT conversation/export process**?

You said:

final game

ChatGPT said:

Okay, this is your **final gameplay recording** for Lab 4. ✅

- **Duration:** 20.74 seconds — meets the 10-second requirement.
- **Purpose:** Use it as your `after.mp4` file.
- **Before video:** Keep your original 12-second recording as `before.mp4`.

I can confirm the duration, but I haven't verified from the video itself that every new feature is visibly demonstrated.

Final steps

1. Rename this recording to `after.mp4`.
2. Keep the original recording as `before.mp4`.
3. Save both in your `Lab-4` submission folder.
4. Save the complete ChatGPT conversation as a PDF.
5. Verify that your four Git commits are pushed to your GitHub repository ↗.

Once the PDF is saved, you'll have all three required deliverables: **before/after videos, updated code, and complete chat history**.

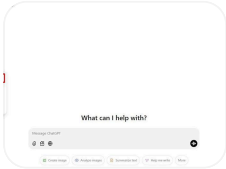
You said:

how to download chat history

ChatGPT said:

The easiest way is to save this ChatGPT conversation as a PDF using Microsoft Edge on your Windows laptop.

Step-by-step instructions



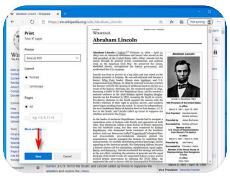
Step 1 — Open this conversation

Stay in this chat containing your Lab 4 work.



Step 2 — Open Print

Press `Ctrl + P` on your keyboard.



Step 3 — Save as PDF

In the print window:

1. Set the printer or destination to **Save as PDF** or **Microsoft Print to PDF**.
2. Click **Print** or **Save**.
3. Choose your `Lab-4` folder.
4. Name the file `chat_history.pdf`.

Step 4 — Verify the PDF

Open the saved PDF and check that your prompts, code changes, AI responses, and all four tasks are included.

Important: Printing may not include the entire conversation if the page only loads part of the chat. Check the PDF before submitting. If it is incomplete, use ChatGPT's account data export instead: go to **Settings** → **Data Controls** → **Export Data**. The export arrives by email and includes your chat history, though it may contain more than this one conversation.

For your assignment, submit the PDF containing this Lab 4 conversation if your instructor requires only the chat used for the task.